Feasibility Study

Resource Coordination Platform for Community Resilience

GROUP NO: 14
PID: 09
MENTOR: MR. ISURU SAMARANAYAKE
7/10/2026

Kumarasinghe H.R.P.R. – 230354T
Lakmal H.M.P. – 230361L
Lakshika G. – 230370M

Contributions

Member	Section
Kumarasinghe H.R.P.R. – 230354T	1, 2.3, 2.4
Lakmal H.M.P. – 230361L	2.5, 3
Lakshika G. – 230370M	2.1, 2.2, 4

Table of contents

1. Introduction 3

1.1 Overview of the Project 3

1.2 Objectives of the Project 3

1.3 The Need for the Project 3

1.4 Overview of Existing Systems and Technologies 4

1.5 Scope of the Project 4

1.6 Deliverables 4

2. Feasibility Study 5

2.1 Financial Feasibility 5

2.2 Technical Feasibility 7

2.3 Resource and Time Feasibility 9

2.3.1 Human Resources 9

2.3.2 Software Resources 9

2.3.3 Hardware Resources 10

2.3.4 Time Estimation 10

2.4 Risk Feasibility 11

2.5 Social/Legal Feasibility 11

3. Considerations 12

4. References 13

1. Introduction

1.1 Overview of the Project

The Resource Coordination Platform for Community Resilience is a comprehensive digital platform designed to improve the coordination of resources, volunteers, and relief activities during emergencies and community crises, such as floods and landslides. The system provides a centralized environment where community members can request assistance, donors can offer resources, volunteers can participate in relief operations, and coordinators can manage all activities through a unified dashboard. By deploying a responsive web application alongside a cross-platform mobile application, the system benefits community organizations by ensuring efficient, transparent, and reliable emergency response management.

1.2 Objectives of the Project

The objectives of this project are to:

- **Design and implement** a centralized platform for coordinating community resources and relief operations during emergency situations.
- **Provide** secure and efficient management of help requests, physical donations, resource inventory, and volunteer activities.
- **Develop** digital tools for coordinators to verify requests, allocate resources accurately, assign volunteers to tasks, and monitor overall operations.
- **Improve** the efficiency, transparency, and reliability of community-based emergency response efforts.

1.3 The Need for the Project

Currently, community organizations often coordinate relief efforts using informal channels such as phone calls, messaging applications, social media, and manual records. These methods frequently lead to duplicated help requests, inefficient resource allocation, and poor coordination of volunteers. Therefore, a centralized platform is critically needed to manage requests, donations, inventory, and volunteer activities effectively. The proposed system resolves these operational bottlenecks, ensuring efficient resource distribution, improved overall coordination, and the protection of sensitive community information during crisis events.

1.4 Overview of Existing Systems and Technologies

Existing disaster management and community coordination platforms, such as Sahana Eden [5], Ushahidi [4], and Crisis Cleanup [6], currently provide functionalities including resource management, volunteer coordination, and incident reporting. However, these existing systems are primarily designed for large-scale organizations and often lack a lightweight, community-focused approach suitable for grassroots initiatives. To address these limitations, the proposed system will be developed using modern, scalable technologies. The organization will utilize software development tools such as React.js [7] for the web interface, Flutter [8] for cross-platform mobile development, and backend development using Node.js [9] with Express.js [10]. Data will be managed using a PostgreSQL [11] database, while real-time communication and security will be handled via Socket.IO [13] and JWT authentication [12].

1.5 Scope of the Project

The scope of the project encompasses several distinct user roles, each with specific functionalities within the system:

- **Coordinators/Administrators:**
  Responsible for managing help requests, verifying data, tracking inventory, allocating resources, assigning tasks to volunteers, and generating administrative reports.
- **Community Members:**
  Capable of registering within the system, submitting requests for assistance, and receiving real-time notifications regarding their requests.
- **Donors:**
  Able to offer resources or physical donations and track the status of their contributions.
- **Volunteers:**
  Provided with functionalities to register for relief activities, view assigned tasks, and participate in coordinated emergency response efforts.

1.6 Deliverables

The primary deliverables of this system include:

- A responsive web application designed for coordinators and administrators to manage emergency response operations.
- A cross-platform mobile application for community members, donors, and volunteers to request assistance, offer resources, and participate in relief activities.
- A secure backend system featuring RESTful APIs, database management, real-time communication capabilities, and role-based access control.

- Comprehensive project documentation, including system requirements, UML diagrams, database designs, testing reports, and user manuals.

2. Feasibility Study

2.1 Financial Feasibility

The proposed Resource Coordination Platform for Community Resilience is considered financially feasible because it can be developed and maintained using cost-effective technologies while providing significant operational benefits to community organizations. The system primarily utilizes open-source software, including React.js [7], Flutter [8], Node.js [9], Express.js [10], PostgreSQL [11], Git [15] and Visual Studio Code, which eliminates software licensing costs and substantially reduces the overall development expenditure.

The hardware required for development consists of standard personal computers and Android mobile devices that are already available to the development team. Therefore, no additional hardware investment is expected during the development phase. Communication and collaboration activities can be carried out using existing internet services and GitHub [16] for version control.

The primary operational expenses are expected to arise from cloud hosting, database hosting, domain registration and internet connectivity. These costs are relatively low and can be scaled according to the number of users and organizational requirements. Since the system follows a modular architecture, future enhancements can be incorporated without significant redevelopment costs, making it a financially sustainable solution.

In addition to minimizing implementation costs, the proposed system is expected to generate indirect financial benefits by reducing manual administrative work, minimizing duplicated resource allocation, improving inventory management and enabling more efficient volunteer coordination. These improvements contribute to better utilization of donated resources while reducing operational inefficiencies during emergency response activities.

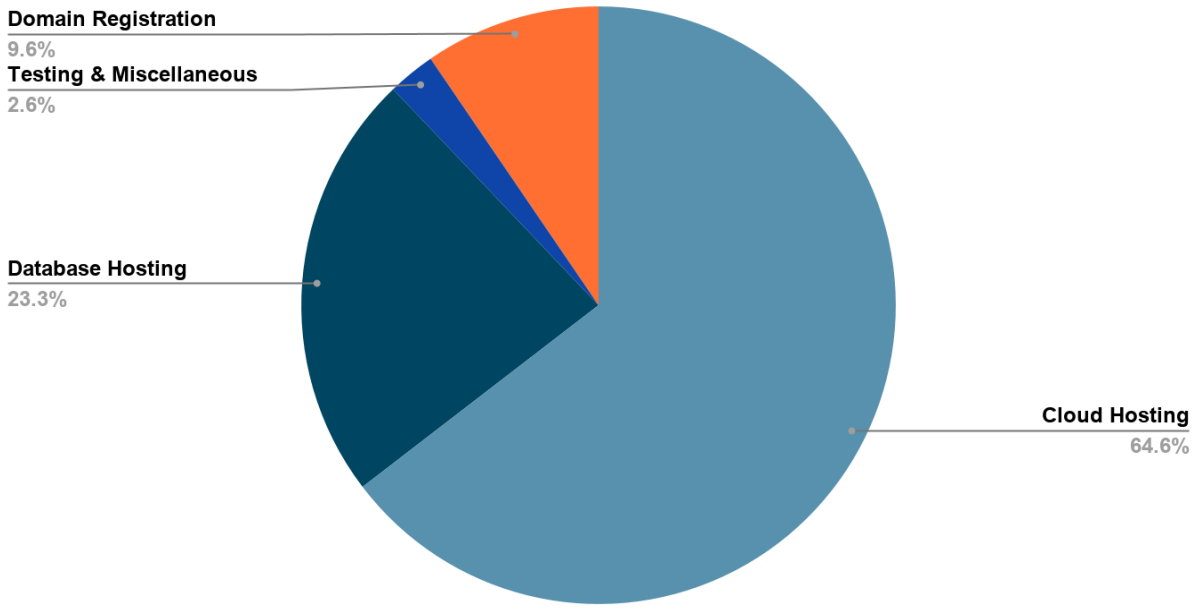
Estimated Development and Operational Costs

Item	Description	Estimated Cost (LKR)
Development Software	React.js, Flutter, Node.js, PostgreSQL, VS Code, GitHub (Open Source)	0
Development Hardware	Existing personal computers and Android devices	0

Internet & Communication	Internet access and online collaboration	0
Domain Registration	Annual domain registration	3,700
Cloud Hosting	Web application and API hosting (annual estimate)	25,000
Database Hosting	PostgreSQL cloud hosting (annual estimate)	9,000
Testing & Miscellaneous	Backups and contingency	1,000
Total Estimated Cost		**38,700**

Note: The estimated costs presented above represent the approximation annual operational expenses for a production/business level deployment. However, for this project, implementation will primarily utilize free and open-source technologies and cloud service free tiers (For example Database hosting will utilize Supabase free tier, Cloud hosting will be supported through AWS promotional credits)

Estimated Cost Distribution



The estimated costs indicate that the proposed system can be developed and deployed with a relatively low financial investment. The use of open-source technologies eliminates licensing expenses, while the remaining operational costs are affordable and scalable. Considering the

expected improvements in resource coordination, transparency, and operational efficiency, the long-term benefits of the system outweigh the implementation and maintenance costs. Therefore, the proposed project is considered financially feasible.

2.2 Technical Feasibility

The proposed Resource Coordination Platform for Community Resilience is technically feasible because it will be developed using modern, reliable, and widely adopted technologies that are well suited for web and mobile application development. The selected technology stack supports the functional and non-functional requirements of the system, including scalability, security, maintainability, and real-time communication.

The system will consist of a web application for coordinators and administrators, a mobile application for community members, donors, and volunteers, and a centralized backend that provides services to both platforms. This architecture ensures consistent data management while reducing development and maintenance efforts by allowing both applications to communicate with a single backend through RESTful APIs [17].

The web application will be developed using React.js [7], which provides a responsive and interactive user interface for managing requests, donations, inventory, volunteer assignments, and reports. The mobile application will be developed using Flutter [8], enabling deployment on both Android and iOS devices from a single codebase while maintaining a consistent user experience across platforms.

The backend will be implemented using Node.js [9] with Express.js [10] and will follow a microservices architecture, where core functionalities such as user management, help request management, donation management, inventory management, volunteer coordination, and notification services are developed as independent services. This modular architecture improves maintainability, scalability, and fault isolation while allowing individual services to be developed, tested, and deployed independently. The event-driven architecture of Node.js makes it well suited for handling multiple concurrent requests during emergency situations. Communication between the frontend applications and backend services will be achieved through RESTful APIs [17], ensuring loose coupling and easy integration between system components.

A PostgreSQL [11] relational database will be used to store and manage user information, help requests, donation records, volunteer details, inventory data, and task assignments. PostgreSQL provides strong transaction management, data integrity, and support for concurrent operations, making it appropriate for applications that require reliable and consistent data management.

System security will be implemented using JSON Web Tokens (JWT) [12] for authentication and role-based access control (RBAC) [18] to ensure that users can only access functions relevant to

their assigned roles. Passwords will be securely encrypted before being stored in the database, reducing the risk of unauthorized access. Secure communication between clients and the server can be achieved using HTTPS when the system is deployed.

The system will also incorporate Socket.IO [13] to support real-time communication between users and the server, enabling immediate updates for help request statuses, volunteer task assignments, inventory changes, and notifications. Apache Kafka [19] will be used as a distributed event-streaming platform to facilitate asynchronous communication between microservices. Kafka improves system reliability by ensuring that events such as new help requests, donation updates, inventory changes, and volunteer assignments are processed efficiently without creating tight dependencies between services. This event-driven approach enhances scalability, fault tolerance, and overall system performance during periods of high user activity. To improve usability in disaster-affected areas, the application will support limited offline data storage, allowing users to continue entering essential information when internet connectivity is unavailable. Once the connection is restored, the stored data will be automatically synchronized with the central database. Additionally, the user interface and API responses will be optimized to operate efficiently under low-bandwidth network conditions.

The proposed architecture also supports multi-tenant logical separation, allowing multiple community organizations or relief groups to use the same application while maintaining separate users, resources, requests, and reports. This approach improves scalability and enables future expansion without requiring separate deployments for each organization.

Proposed Technologies

Component	**Technology**	**Purpose**
Web Application	React.js	Responsive user interface for administrators and coordinators
Mobile Application	Flutter	Cross-platform mobile application for community members, donors, and volunteers
Backend	Node.js with Express.js	Business logic and RESTful API services
Database	PostgreSQL	Storage and management of application data
Authentication	JSON Web Token (JWT)	Secure user authentication and session management
Authorization	Role-Based Access Control (RBAC)	Restrict system access based on user roles

Real-Time Communication	Socket.IO	Live notifications and status updates
Event Streaming / Messaging	Apache Kafka	Asynchronous communication and event processing between microservices
API Architecture	RESTful APIs	Communication between frontend applications and backend
Version Control	Git & GitHub	Source code management and collaboration
Deployment	Docker	Containerized deployment and scalability

The selected technologies are mature, well documented, and supported by large developer communities, ensuring long-term maintainability and future extensibility of the proposed system.

2.3 Resource and Time Feasibility

2.3.1 Human Resources

The project will be developed by a three-member software development team. Responsibilities are distributed among team members according to the major components of the system to ensure efficient development, integration, testing, and documentation. This division of work improves productivity while allowing collaborative development throughout the project lifecycle.

Human Resource	**Responsibility**
Team Member 1	Backend services, database design, API development
Team Member 2	Web application development and user interface
Team Member 3	Mobile application development, testing, and documentation

2.3.2 Software Resources

The following software resources will be used to support development, collaboration, testing, and project management activities.

Software Resource	**Purpose**
Visual Studio Code	Source code development

Git & GitHub	Version control and team collaboration
GitHub Projects	Project planning and task management
Apache Kafka	Messaging
Docker Desktop	Local development and containerized testing
AWS EC2 / Railway	For deployment of backend services
Zoom / WhatsApp	Team communication and meetings

2.3.3 Hardware Resources

Hardware Resource	**Purpose**
Personal Computers/ Laptops	Software development and testing
Android Smartphones	Mobile application testing in developer mode
Cloud Hosting Environment	For hosting backend, Kafka, databases

2.3.4 Time Estimation

The project is planned to be completed before 30 September 2026.

Week	**Planned Activities**
Week 1	Requirement analysis, project proposal, GitHub repository and project setup
Week 2	Feasibility study, user stories, system architecture and API endpoint contract
Week 3	Database design, UI/ UX design and project environment setup
Weeks 4–6	Development of core microservices, Configuring Kafka, implement JWT security and database interactions.
Week 7	Integration of web application, mobile application, backend services and database
Week 8	System testing, performance optimization and bug fixing.
Week 9	Documentation, deployment preparation and user acceptance testing.
Week 10	Final report completion and system deployment.

2.4 Risk Feasibility

The following are some possible risks that may arise and the current plan to overcome those:

- **Network Unreliability:**
  Disaster-affected areas frequently experience poor or unavailable internet connectivity. To mitigate this risk, the system incorporates offline data synchronization, allowing users to input critical data into local storage that will automatically synchronize with the central database once a connection is re-established. Additionally, the platform is optimized to operate under low-bandwidth conditions.
- **System Overload During Crises:**
  Emergency situations can lead to sudden and unpredictable spikes in user traffic. This risk is mitigated by using a scalable microservices architecture, load balancing, application-level caching and optimized database queries.
- **Data Security Breaches:**
  The platform handles sensitive community information, including personal details of victims, volunteers and donors. This risk is addressed by implementing JWT for secure authentication and strict role-based access control (RBAC) [18] to ensure that users can only access data relevant to their authorization level.
- **Duplicate or False Help Requests:**
  Coordinators will verify submitted requests before approval to prevent fraudulent or duplicated requests from entering the allocation process.
- **Inter service communication failures:**
  Communication failures between system components will be minimized through reliable RESTful APIs, proper exception handling, and retry mechanisms.
- **Scope Creep:**
  Managing feature requirements within a strict development timeline can be challenging. This is mitigated by defining a clear project scope that focuses strictly on core deliverables, such as help request management, inventory tracking, and volunteer assignment and maintaining a structured development schedule.

2.5 Social/Legal Feasibility

The proposed Resource Coordination Platform for Community Resilience is highly feasible from both social and legal perspectives. Socially, the system addresses a critical need by improving the coordination of resources, volunteers, and relief activities during natural disasters such as floods and landslides. By transitioning away from inefficient manual records and disjointed communication channels, the platform fosters community resilience and improves the transparency and reliability of grassroots emergency response efforts. It directly empowers

community members to request assistance and allows volunteers and donors to participate effectively in relief activities through a centralized platform.
Legally and ethically, the system prioritizes the protection of sensitive community information during emergencies. The system will also comply with applicable privacy regulations by collecting only the information necessary for emergency response and maintaining audit logs of administrative activities. The architecture strictly enforces data privacy through role-based access controls, ensuring that only authorized coordinators and administrators can view sensitive help requests or resource allocations. Furthermore, the project utilizes a technology stack comprised entirely of open-source software (such as React.js [7], Flutter [8], and PostgreSQL [11]), which ensures full compliance with standard software licensing regulations and eliminates the risk of intellectual property infringement.

3. Considerations

During the design and implementation of the Resource Coordination Platform for Community Resilience, several critical factors must be prioritized to ensure the system operates effectively under crisis conditions.

- **Usability**:
  The application will be deployed in high-stress, disaster-affected environments. The provision of a responsive web application for coordinators and a cross-platform mobile application for community members ensures intuitive navigation, a minimal learning curve, and broad accessibility across different devices.
- **Performance and system resilience:**
  These are vital concerns, given the likelihood of degraded infrastructure in disaster zones. To support continuous operation, features such as offline data synchronization and optimization for low-bandwidth environments and high user traffic during emergencies are prioritized to guarantee functionality when network connectivity is heavily degraded or unreliable.
- **Security and data privacy:**
  Remain paramount to safeguard sensitive community information. The implementation of secure backend APIs, JWT authentication, and robust role-based access controls ensures that unauthorized access is strictly prevented and data integrity is maintained at all times.
- **Scalability:**
  It is addressed by designing the system architecture to support multi-tenant organizations. This essential feature allows multiple independent relief groups and community organizations to operate simultaneously on the same platform without data overlap, ensuring seamless expansion and long-term sustainability during large-scale emergency response efforts.

- **Maintainability:**
  The system follows a modular architecture that simplifies maintenance, testing, and future feature enhancements while reducing the impact of changes on existing components.

4. References

[1] H. Betke et al., "A Design Theory for Spontaneous Volunteer Coordination Systems in Disaster Response," Proceedings of the Hawaii International Conference on System Sciences, 2024. Available: https://scholarspace.manoa.hawaii.edu/items/6e1f845c-7e16-492b-b48d-c4a6bd4f09e5 (Accessed: 02-Jul-2026)

[2] D. A. Carpenter et al., "Resource exchange patterns between Voluntary Organizations Active in Disaster (VOADs): A multilevel network assessment to improve disaster response capacity," International Journal of Disaster Risk Reduction, vol. 108, 2024.

[3] M. Sperlinga, "Decision support for disaster relief: Coordinating spontaneous volunteers," European Journal of Operational Research, vol. 299, no. 2, pp. 690-705, 2022.

[4] Ushahidi, "Crowdsourcing Solutions to Empower Communities." Available: https://www.ushahidi.com/ (Accessed: 01-Jul-2026).

[5] Sahana Software Foundation, "Sahana Eden Disaster Management Platform." Available: https://sahanafoundation.org/eden/ (Accessed: 01-Jul-2026).

[6] Crisis Cleanup, "Crisis Cleanup Platform." Available: https://api.crisiscleanup.org/ (Accessed: 01-Jul-2026).

[7] React Documentation. Available: https://react.dev/ (Accessed: 12-Jul-2026).

[8] Flutter Documentation. Available: https://docs.flutter.dev/ (Accessed: 12-Jul-2026).

[9] Node.js Documentation. Available: https://nodejs.org/docs/latest/api/ (Accessed: 12-Jul-2026).

[10] Express.js Documentation. Available: https://expressjs.com/ (Accessed: 12-Jul-2026).

[11] PostgreSQL Global Development Group. PostgreSQL Documentation. Available: https://www.postgresql.org/docs/ (Accessed: 12-Jul-2026).

[12] JSON Web Tokens. Available: https://jwt.io/introduction (Accessed: 12-Jul-2026).

[13] Socket.IO Documentation. Available: https://socket.io/docs/ (Accessed: 12-Jul-2026).

[14] Docker Documentation. Available: https://docs.docker.com/ (Accessed: 12-Jul-2026).

[15] Git Documentation. Available: https://git-scm.com/doc (Accessed: 12-Jul-2026).

[16] GitHub Documentation. Available: https://docs.github.com/ (Accessed: 12-Jul-2026).

[17] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000. Available: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm (Accessed: 12-Jul-2026).

[18] R. Sandhu, E. Coyne, H. Feinstein and C. Youman, "Role-Based Access Control Models," IEEE Computer, vol. 29, no. 2, pp. 38–47, 1996.

[19] Apache Kafka Documentation. Available: https://kafka.apache.org/documentation/ (Accessed: 12-Jul-2026).